RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Data Science

V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Name: Praveen Kumar S

Ex 6b: Time Series Data Analysis and Forecast

Register Number: 241801212 **Date:** 02-10-2026

Dataset: Airline Passengers (jbrownlee/Datasets, GitHub) **Tools:** Python, Pandas, statsmodels, scikit-learn, Matplotlib

Aim

To analyse the monthly airline passenger time series, check its stationarity using the Augmented Dickey-Fuller test, make it stationary by differencing, build an ARIMA model to forecast passenger numbers for the test period, and evaluate the forecast accuracy.

Task 1: Download the airline-passengers dataset

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import itertools

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

import warnings
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')

# Source: https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv
# (downloaded and saved in the working folder as airline-passengers.csv)
print("Libraries imported successfully")
```

Out [1]:

```
Libraries imported successfully
```

Task 2: Read the CSV with parse_dates and index_col

In [2]:

```
df = pd.read_csv('airline-passengers.csv', parse_dates=['Month'], index_col='Month')
df = df.asfreq('MS')            # monthly data, each date is the start of the month

print("Index type :", type(df.index).__name__)
print("Frequency :", df.index.freqstr)
df.head()
```

Out [2]:

```
Index type : DatetimeIndex
Frequency  : MS
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

5 rows × 1 columns

Task 3: Explore the dataset

In [3]:

```
df.tail()
```

Out [3]:

Month	Passengers
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

5 rows × 1 columns

In [4]:

```
print("Shape          :", df.shape)
print("Date range     :", df.index.min().strftime('%b %Y'), "to", df.index.max().strftime('%b %Y'))
print("Missing values:", df['Passengers'].isnull().sum())
print("Duplicate dates:", df.index.duplicated().sum())
print()
df.info()
```

Out [4]:

```
Shape          : (144, 1)
Date range     : Jan 1949 to Dec 1960
Missing values: 0
Duplicate dates: 0

<class 'pandas.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Freq: MS
Data columns (total 1 columns):
 #    Column       Non-Null Count  Dtype
---  -------      --------------  -----
 0    Passengers  144 non-null     int64
dtypes: int64(1)
memory usage: 2.2 KB
```

In [5]:

```
df.describe().T.round(2)
```

Out [5]:

	count	mean	std	min	25%	50%	75%	max
Passengers	144.0	280.3	119.97	104.0	180.0	265.5	360.5	622.0

1 rows × 8 columns

In [6]:

```
# Total passengers per year and growth
yearly = df['Passengers'].resample('YS').sum()
yearly.index = yearly.index.year
growth = pd.DataFrame({'Total Passengers': yearly,
                       'Growth (%)': (yearly.pct_change() * 100).round(1)})
growth.T
```

Out [6]:

Month	1949	1950	1951	1952	1953	1954	1955	1956	1957	1958	1959	1960
Total Passengers	1520.0	1676.0	2042.0	2364.0	2700.0	2867.0	3408.0	3939.0	4421.0	4572.0	5140.0	5714.0
Growth (%)	NaN	10.3	21.8	15.8	14.2	6.2	18.9	15.6	12.2	3.4	12.4	11.2

2 rows × 12 columns

> **Observation:** The dataset has 144 monthly values from January 1949 to December 1960 with no missing values or duplicate dates. Passenger numbers range from 104 to 622 thousand. Yearly totals grew every year, from 1,520 thousand in 1949 to 5,714 thousand in 1960, nearly four times higher.

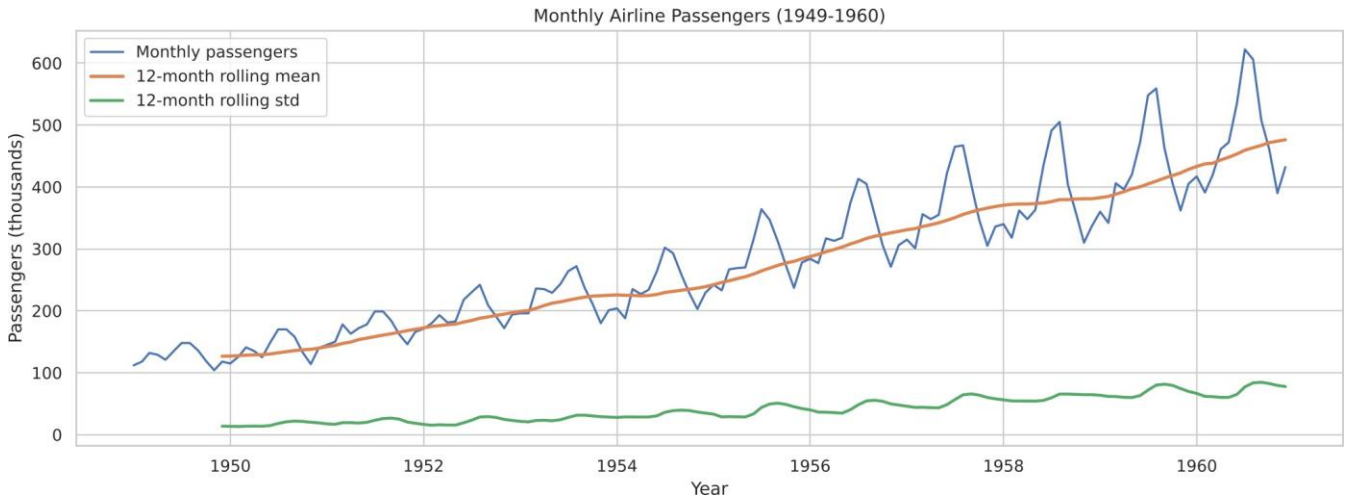
Task 4: Plot the series to inspect trend, seasonality and outliers

In [7]:

```
series = df['Passengers']

plt.figure(figsize=(13, 5))
plt.plot(series, color='#4C72B0', label='Monthly passengers')
plt.plot(series.rolling(12).mean(), color='#DD8452', linewidth=2.2, label='12-month rolling mean')
plt.plot(series.rolling(12).std(), color='#55A868', linewidth=2, label='12-month rolling std')
plt.title('Monthly Airline Passengers (1949-1960)')
plt.xlabel('Year')
plt.ylabel('Passengers (thousands)')
plt.legend()
plt.tight_layout()
plt.show()
```

Out [7]:

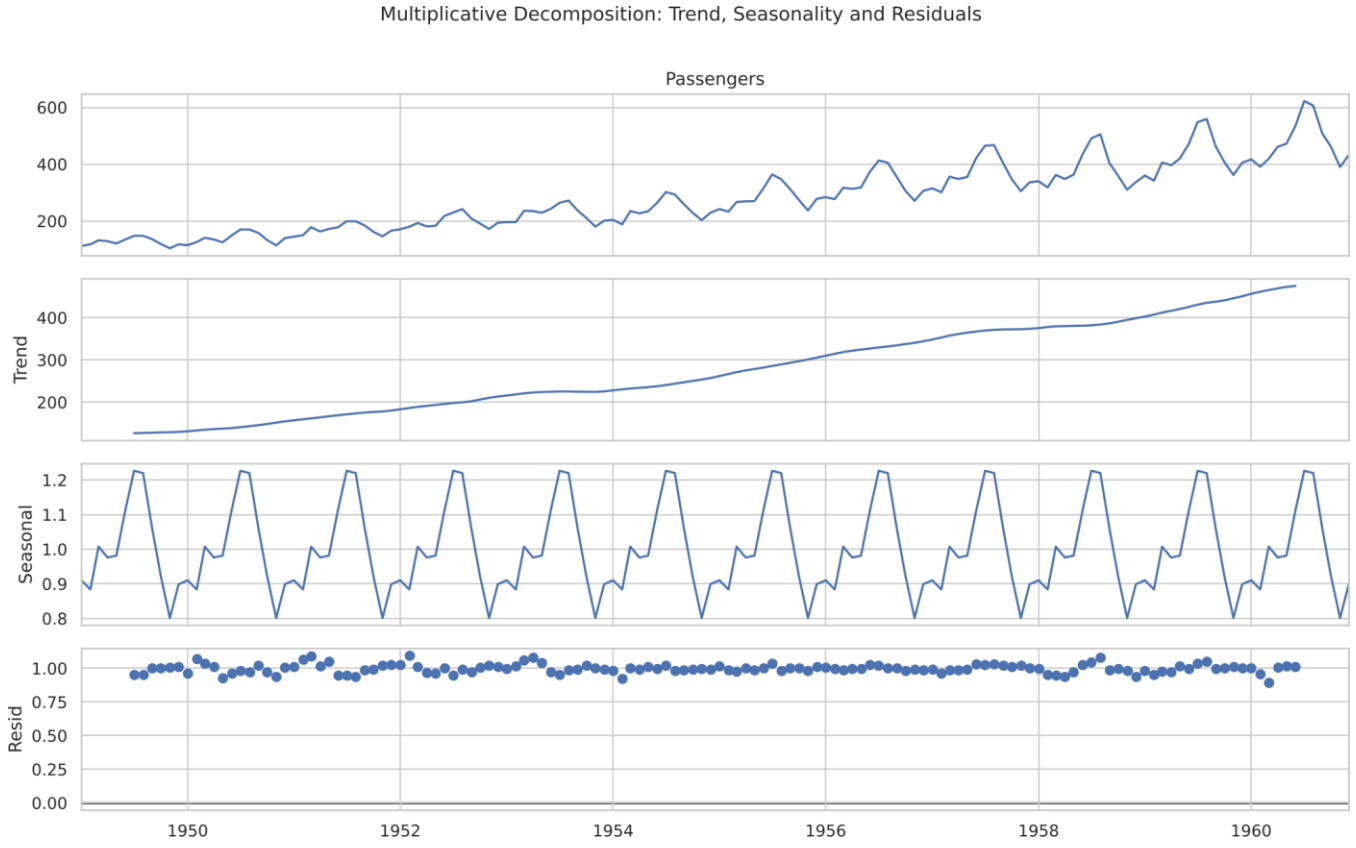


In [8]:

```
decomposition = seasonal_decompose(series, model='multiplicative', period=12)

fig = decomposition.plot()
fig.set_size_inches(13, 8)
fig.suptitle('Multiplicative Decomposition: Trend, Seasonality and Residuals', y=1.01, fontsize=13)
plt.tight_layout()
plt.show()
```

Out [8]:



In [9]:

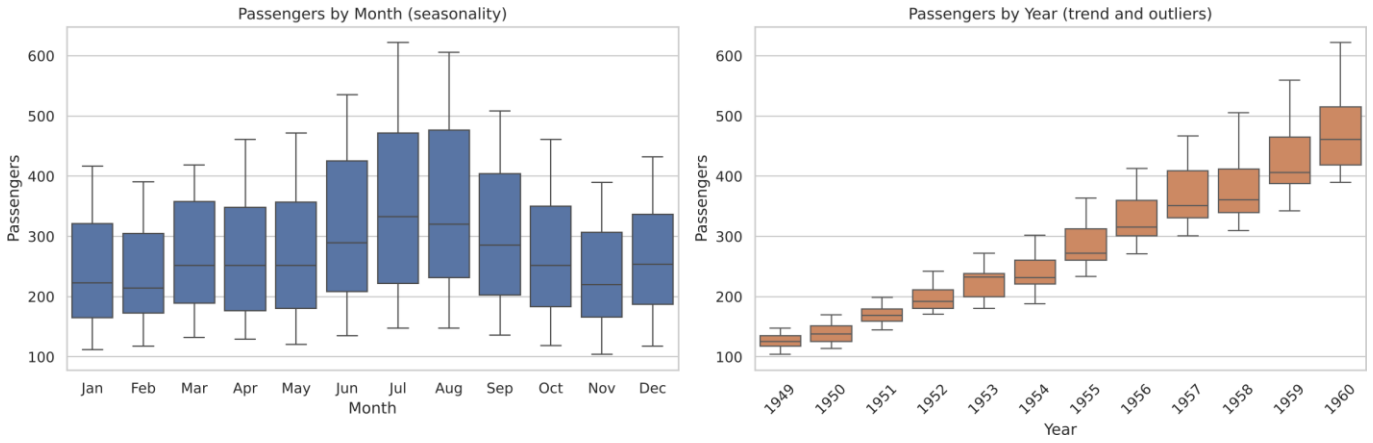
```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
monthly = df.assign(Month=df.index.strftime('%b'), Year=df.index.year)

sns.boxplot(data=monthly, x='Month', y='Passengers', color='#4C72B0', ax=axes[0])
axes[0].set_title('Passengers by Month (seasonality)')

sns.boxplot(data=monthly, x='Year', y='Passengers', color='#DD8452', ax=axes[1])
axes[1].set_title('Passengers by Year (trend and outliers)')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

Out [9]:



Observation: The series has a clear upward **trend** and strong yearly **seasonality**: passenger numbers peak every July–August (average about 351 thousand) and are lowest in November and February. The rolling standard deviation and the size of the seasonal swings grow over time, so the seasonality is multiplicative. The box plots show no extreme outliers; the highest values are summer peaks such as July 1960 (622 thousand).

Task 5: Stationarity check using the Augmented Dickey-Fuller (ADF) test

- Null hypothesis (H0): the series has a unit root, i.e. it is **non-stationary**.
- If the p-value is less than 0.05, H0 is rejected and the series is **stationary**.

In [10]:

```
def adf_test(data, title):
    result = adfuller(data.dropna(), autolag='AIC')
    print(f"ADF test - {title}")
    print(f"  ADF statistic        : {result[0]:.4f}")
    print(f"  p-value              : {result[1]:.4f}")
    print(f"  Lags used            : {result[2]}")
    print(f"  Observations used    : {result[3]}")
    for level, value in result[4].items():
        print(f"  Critical value {level:>4s}: {value:.4f}")
    stationary = result[1] < 0.05
    print("  Conclusion           :", "Stationary" if stationary else "Non-stationary")
    return result[1]

p_original = adf_test(series, "Original series")
```

Out [10]:

```
ADF test - Original series
  ADF statistic        : 0.8154
  p-value              : 0.9919
  Lags used            : 13
  Observations used : 130
  Critical value    1%: -3.4817
  Critical value    5%: -2.8840
  Critical value  10%: -2.5788
  Conclusion           : Non-stationary
```

> **Observation:** The p-value (0.9919) is far above 0.05, so the null hypothesis cannot be rejected. The original series is **non-stationary**, as expected from its trend and seasonality.

Task 6: Convert the series to stationary by differencing (d = 1)

In [11]:

```
diff_1 = series.diff().dropna()         # y(t) - y(t-1)

print("First-differenced series (first 5 values):")
print(diff_1.head())
```

Out [11]:

```
First-differenced series (first 5 values):
Month
1949-02-01      6.0
1949-03-01     14.0
1949-04-01     -3.0
1949-05-01     -8.0
1949-06-01     14.0
Freq: MS, Name: Passengers, dtype: float64
```

Task 7: Re-run the ADF test on the differenced series and visualise it

In [12]:

```
p_diff1 = adf_test(diff_1, "First difference (d = 1)")
```

Out [12]:

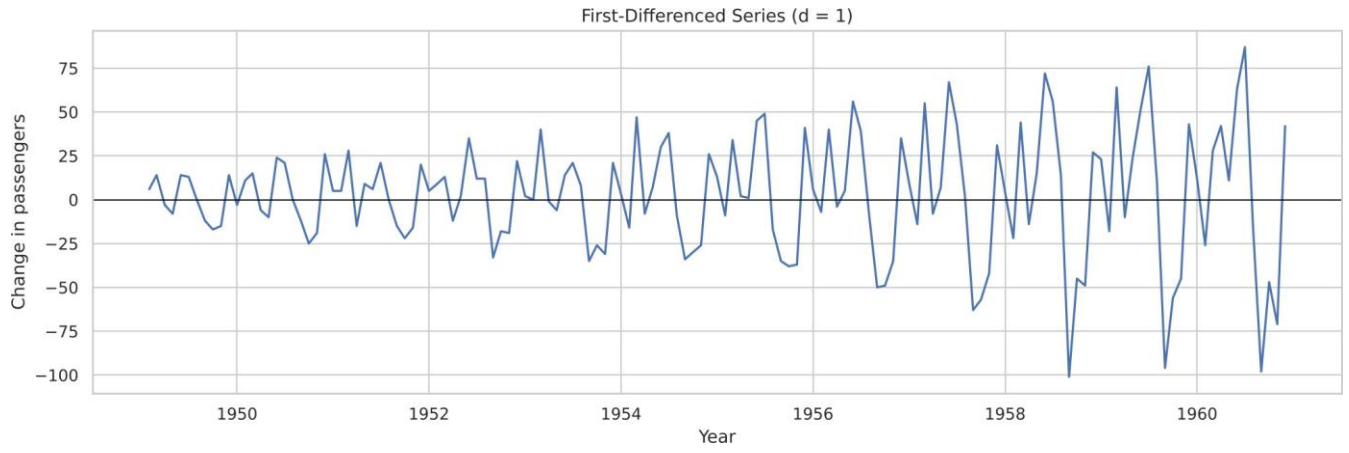
```
ADF test - First difference (d = 1)
  ADF statistic        : -2.8293
  p-value              : 0.0542
  Lags used            : 12
  Observations used : 130
  Critical value    1%: -3.4817
  Critical value    5%: -2.8840
  Critical value  10%: -2.5788
  Conclusion           : Non-stationary
```

> **Observation:** Differencing removed the trend and the p-value dropped to 0.0542, but it is still above 0.05, so the first-differenced series is **still non-stationary**. The plot below also shows that the size of the fluctuations grows over time.

In [13]:

```
plt.figure(figsize=(13, 4.5))
plt.plot(diff_1, color='#4C72B0')
plt.axhline(0, color='black', linewidth=0.8)
plt.title('First-Differenced Series (d = 1)')
plt.xlabel('Year')
plt.ylabel('Change in passengers')
plt.tight_layout()
plt.show()
```

Out [13]:



Task 8: Second difference (d = 2) or log transformation

In [14]:

```
# Option 1: log transformation followed by one difference
log_diff_1 = np.log(series).diff().dropna()
p_logdiff = adf_test(log_diff_1, "Log transform + first difference")
print()

# Option 2: second difference of the original series
diff_2 = series.diff().diff().dropna()
p_diff2 = adf_test(diff_2, "Second difference (d = 2)")
```

Out [14]:

```
ADF test - Log transform + first difference
  ADF statistic        : -2.7171
  p-value              : 0.0711
  Lags used            : 14
  Observations used    : 128
  Critical value   1%: -3.4825
  Critical value   5%: -2.8844
  Critical value  10%: -2.5790
  Conclusion           : Non-stationary

ADF test - Second difference (d = 2)
  ADF statistic        : -16.3842
  p-value              : 0.0000
  Lags used            : 11
  Observations used    : 130
  Critical value   1%: -3.4817
  Critical value   5%: -2.8840
  Critical value  10%: -2.5788
  Conclusion           : Stationary
```

In [15]:

```
summary = pd.DataFrame({
    'Series': ['Original', 'First difference (d=1)', 'Log + first difference', 'Second difference
(d=2)'],
    'p-value': [p_original, p_diff1, p_logdiff, p_diff2]})
summary['Stationary (p < 0.05)'] = np.where(summary['p-value'] < 0.05, 'Yes', 'No')
summary['p-value'] = summary['p-value'].round(4)
summary.set_index('Series')
```

Out [15]:

Series	p-value	Stationary (p < 0.05)
Original	0.9919	No
First difference (d=1)	0.0542	No
Log + first difference	0.0711	No
Second difference (d=2)	0.0000	Yes

4 rows × 2 columns

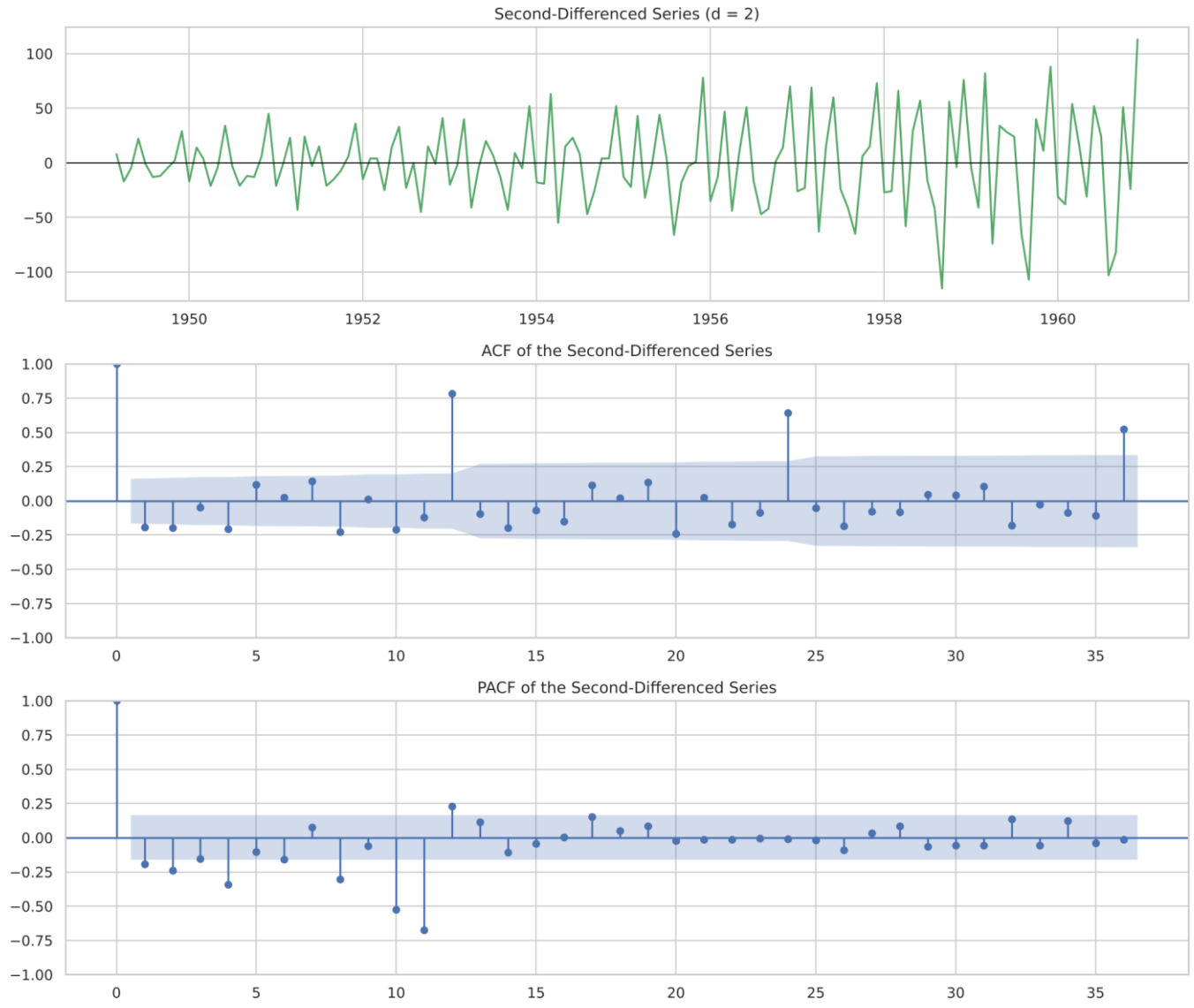
> **Observation:** The log transformation followed by one difference is still non-stationary (p = 0.0711). The second difference gives a p-value of 0.0000, so the series becomes **stationary with d = 2**. Therefore d = 2 is used in the ARIMA model.

In [16]:

```
d = 2   # number of differences needed to make the series stationary

fig, axes = plt.subplots(3, 1, figsize=(13, 11))
axes[0].plot(diff_2, color='#55A868')
axes[0].axhline(0, color='black', linewidth=0.8)
axes[0].set_title('Second-Differenced Series (d = 2)')
plot_acf(diff_2, lags=36, ax=axes[1], title='ACF of the Second-Differenced Series')
plot_pacf(diff_2, lags=36, ax=axes[2], method='ywm', title='PACF of the Second-Differenced Series')
plt.tight_layout()
plt.show()
```

Out [16]:



Observation: The second-differenced series varies around a constant mean of zero. The ACF has large spikes at lags 12, 24 and 36, and the PACF has significant spikes up to lags 11–12. This reflects the yearly seasonal pattern and is why AR terms up to lag 12 are considered for the model.

Task 9: Split the original series into training (80%) and testing (20%) sets

In [17]:

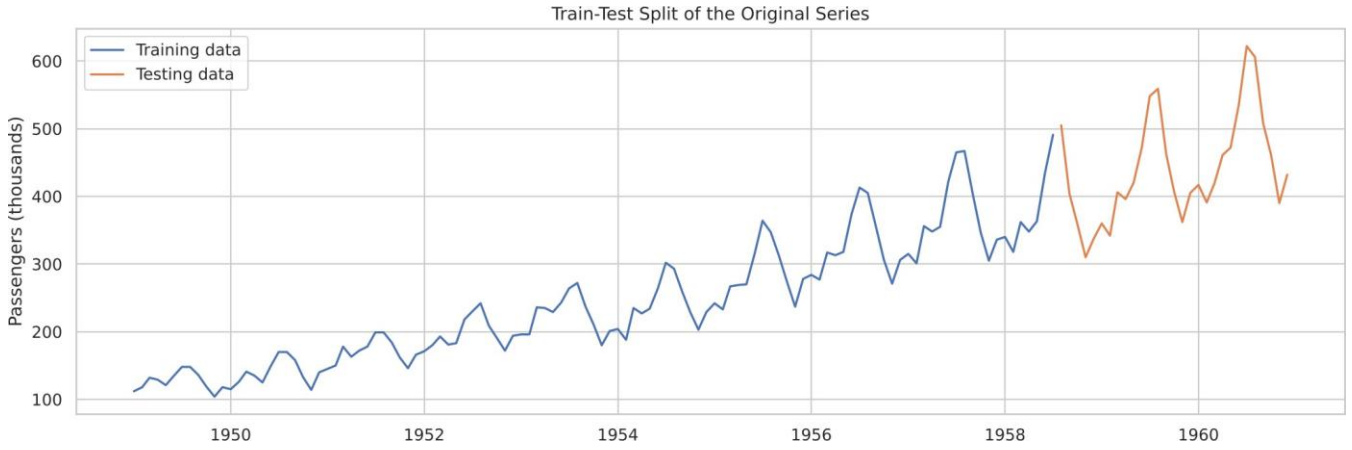
```
split = int(len(series) * 0.8)
train, test = series.iloc[:split], series.iloc[split:]

print(f"Training set: {len(train)} months ({train.index[0]:%b %Y} to {train.index[-1]:%b %Y})")
print(f"Testing set : {len(test)} months ({test.index[0]:%b %Y} to {test.index[-1]:%b %Y})")

plt.figure(figsize=(13, 4.5))
plt.plot(train, color='#4C72B0', label='Training data')
plt.plot(test, color='#DD8452', label='Testing data')
plt.title('Train-Test Split of the Original Series')
plt.ylabel('Passengers (thousands)')
plt.legend()
plt.tight_layout()
plt.show()
```

Out [17]:

```
Training set: 115 months (Jan 1949 to Jul 1958)
Testing set : 29 months (Aug 1958 to Dec 1960)
```



Task 10: Fit the ARIMA model on the training set (d = 2)

The PACF has significant spikes up to lag 12 because of the yearly seasonality. The values of p and q are therefore chosen by fitting ARIMA(p, 2, q) models on the **training set only** and selecting the one with the lowest AIC (Akaike Information Criterion).

In [18]:

```
candidates = []
for p, q in itertools.product([0, 1, 2, 4, 6, 8, 10, 12], [0, 1, 2]):
    fitted = ARIMA(train, order=(p, d, q)).fit(method_kwargs={'maxiter': 1000})
    candidates.append({'Order (p, d, q)': (p, d, q), 'AIC': round(fitted.aic, 2)})

candidates = pd.DataFrame(candidates).sort_values('AIC').reset_index(drop=True)
best_order = candidates.loc[0, 'Order (p, d, q)']
print("Models compared:", len(candidates))
print("Best order by AIC:", best_order)
candidates.head(8)
```

Out [18]:

```
Models compared: 24
Best order by AIC: (12, 2, 0)
```

	Order (p, d, q)	AIC
0	(12, 2, 0)	870.32
1	(12, 2, 1)	871.19
2	(12, 2, 2)	871.23
3	(10, 2, 2)	920.61
4	(10, 2, 1)	943.58
5	(8, 2, 2)	996.19
6	(10, 2, 0)	1023.27
7	(8, 2, 1)	1028.14

8 rows × 2 columns

In [19]:

```
model = ARIMA(train, order=best_order)
model_fit = model.fit(method_kwargs={'maxiter': 1000})
print(model_fit.summary())
```

Out [19]:

```
                               SARIMAX Results
==============================================================================
Dep. Variable:              Passengers   No. Observations:                  115
Model:                ARIMA(12, 2, 0)   Log Likelihood                -422.158
Date:                Thu, 01 Oct 2026   AIC                            870.316
Time:                        19:52:39   BIC                            905.772
Sample:                    01-01-1949   HQIC                           884.703
                         - 07-01-1958
Covariance Type:                  opg
==========  =======================  =========================  ==========
                 coef    std err          z      P>|z|      [0.025      0.975]
----------  -----------  ---------  ---------  ---------  ---------  ---------
ar.L1         -1.1630      0.092    -12.620      0.000      -1.344      -0.982
ar.L2         -1.2169      0.096    -12.657      0.000      -1.405      -1.028
ar.L3         -1.1403      0.099    -11.504      0.000      -1.335      -0.946
ar.L4         -1.1941      0.092    -12.951      0.000      -1.375      -1.013
ar.L5         -1.1083      0.103    -10.806      0.000      -1.309      -0.907
ar.L6         -1.1477      0.094    -12.243      0.000      -1.331      -0.964
ar.L7         -1.1124      0.101    -10.996      0.000      -1.311      -0.914
ar.L8         -1.1988      0.104    -11.563      0.000      -1.402      -0.996
ar.L9         -1.1407      0.099    -11.545      0.000      -1.334      -0.947
ar.L10        -1.2270      0.101    -12.161      0.000      -1.425      -1.029
ar.L11        -1.1228      0.106    -10.544      0.000      -1.331      -0.914
ar.L12        -0.2615      0.112     -2.334      0.020      -0.481      -0.042
sigma2        82.0982     12.774      6.427      0.000      57.061     107.135
===================================  ==========  ========================  ================
Ljung-Box (L1) (Q):                   0.05   Jarque-Bera (JB):                 2.55
Prob(Q):                              0.83   Prob(JB):                         0.28
Heteroskedasticity (H):               1.91   Skew:                             0.37
Prob(H) (two-sided):                  0.05   Kurtosis:                         3.03
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

> **Observation:** ARIMA(12, 2, 0) has the lowest AIC (870.32). All 12 AR coefficients are statistically significant ($p < 0.05$). The Ljung-Box test (Prob(Q) = 0.83) shows no autocorrelation left in the residuals, and the Jarque-Bera test (Prob(JB) = 0.28) shows the residuals are approximately normal, so the model fits the training data well.

Task 11: Forecast the test period

In [20]:

```
forecast_result = model_fit.get_forecast(steps=len(test))
predictions = forecast_result.predicted_mean
conf_int = forecast_result.conf_int(alpha=0.05)

comparison = pd.DataFrame({'Actual': test,
                           'Predicted': predictions.round(1),
                           'Error': (test - predictions).round(1)})
comparison.index = comparison.index.strftime('%b %Y')
comparison.head(12)
```

Out [20]:

	Actual	Predicted	Error
Aug 1958	505	490.9	14.1
Sep 1958	404	434.3	-30.3
Oct 1958	359	376.8	-17.8
Nov 1958	310	338.0	-28.0
Dec 1958	337	367.4	-30.4
Jan 1959	360	367.1	-7.1
Feb 1959	342	345.3	-3.3
Mar 1959	406	379.2	26.8
Apr 1959	396	368.3	27.7
May 1959	420	385.4	34.6
Jun 1959	472	457.9	14.1
Jul 1959	548	513.6	34.4

12 rows × 3 columns

Task 12: Evaluation metrics (MSE, MAE, RMSE)

In [21]:

```
mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((test - predictions) / test)) * 100

print(f"ARIMA{best_order} forecast accuracy on the test set")
print(f" MSE  : {mse:.2f}")
print(f"  MAE   : {mae:.2f}")
print(f"  RMSE  : {rmse:.2f}")
print(f" MAPE  : {mape:.2f}%    (average error as a percentage of actual passengers)")
print(f"\nMean of the test data: {test.mean():.2f} thousand passengers")
```

Out [21]:

```
ARIMA(12, 2, 0) forecast accuracy on the test set
  MSE  : 1238.88
  MAE   : 27.34
  RMSE  : 35.20
  MAPE  : 5.93%    (average error as a percentage of actual passengers)

Mean of the test data: 440.31 thousand passengers
```

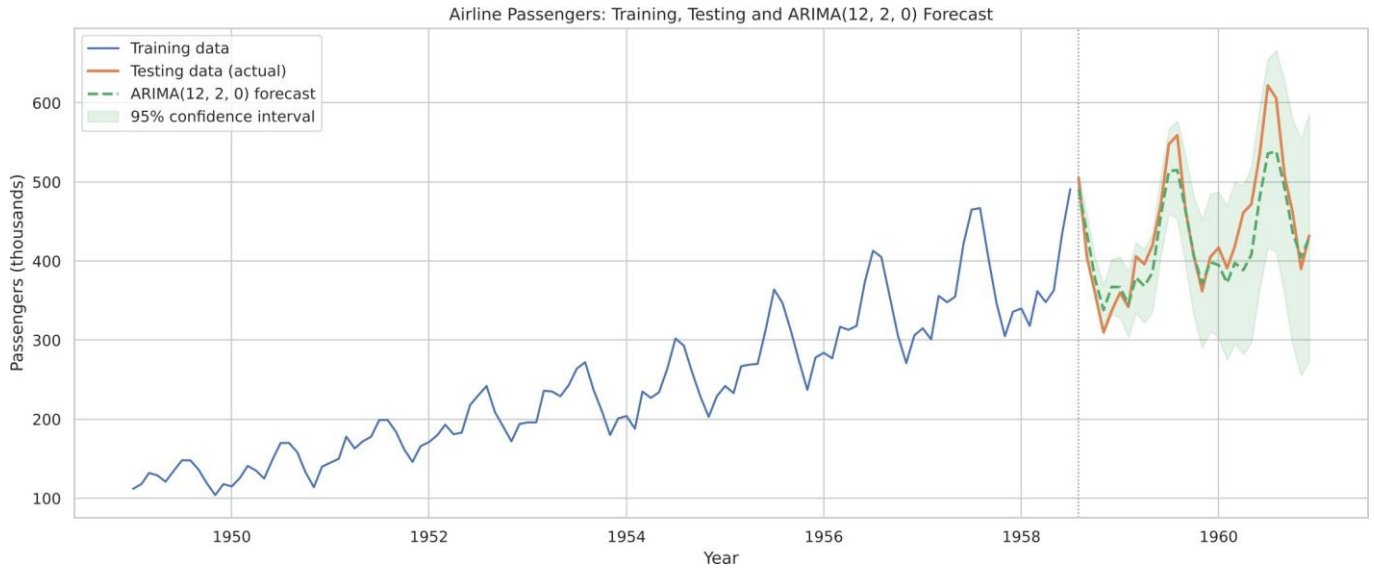
> **Observation:** On the 29-month test period the model has an MAE of 27.34 and an RMSE of 35.20 thousand passengers. Compared with the average test value of 440.31 thousand, the forecasts are off by only about 5.93% on average (MAPE).

Task 13: Train, test and predictions in a single plot

In [22]:

```
plt.figure(figsize=(14, 6))
plt.plot(train, color='#4C72B0', label='Training data')
plt.plot(test, color='#DD8452', linewidth=2, label='Testing data (actual)')
plt.plot(predictions, color='#55A868', linewidth=2, linestyle='--', label=f'ARIMA{best_order}
forecast')
plt.fill_between(conf_int.index, conf_int.iloc[:, 0], conf_int.iloc[:, 1],
                 color='#55A868', alpha=0.15, label='95% confidence interval')
plt.axvline(test.index[0], color='gray', linestyle=':', linewidth=1)
plt.title(f'Airline Passengers: Training, Testing and ARIMA{best_order} Forecast')
plt.xlabel('Year')
plt.ylabel('Passengers (thousands)')
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()
```

Out [22]:



Observation: The forecast follows both the upward trend and the yearly seasonal pattern of the test data, with the summer peaks and winter dips in the right months. 28 of the 29 actual values fall inside the 95% confidence interval (the exception is September 1958), and the interval becomes wider further into the future as uncertainty grows.

Result

Thus the monthly airline passenger data (1949–1960) was analysed and found to have an upward trend and yearly seasonality. The ADF test showed that the original series was non-stationary, and it became stationary after second-order differencing (d = 2). An ARIMA(12, 2, 0) model, selected by AIC, was fitted on the training data and used to forecast the test period with an MSE of 1238.88, an MAE of 27.34 and an RMSE of 35.20 (MAPE 5.93%).